AutoFile Manager

Automated Android File & Storage Management System Application

JAVA – XML / ROOM -ANDROID STUDIO

Step1 : PROJECT TITLE :

AutoFile Manager : Automated Android File & Storage Management System Application.

PROJECT IDEA

AutoFile Manager is an Android application designed to monitor mobile storage and manage files automatically and based on user-defined conditions.

Step 2: Project Synopsis

Title

AutoFile Manager – Automated Android File & Storage Management System

Introduction

AutoFile Manager is an Android application that automatically analyzes mobile storage and helps users manage files and applications.

The application continuously monitors storage usage. When storage reaches **90% or above**, the application sends a notification to the user.

The user can select whether to start a storage analysis.

After confirmation, the application scans the available files and generates storage-management recommendations.

The application can:

- Detect duplicate files.
- Detect large files.
- Detect old files.
- Delete files based on a user-defined period (days , months) .
- Detect potentially unused applications.
- Analyze large files for compression.
- Compress supported files into ZIP archives.
- Calculate storage savings.
- Display cleanup recommendations.
- Generate storage reports.

Inside AutoFile Manager:

- **Auto Manage — ON/OFF**
- All
- Duplicate Files
- Junk/Cache Files
- Unused Apps
- Large Files
- Old Files
- Downloads
- Cleanup conditions
- Automatic background monitoring
- Notifications when action is needed

The user can define the periods of files to be included in automatic cleanup.

For example:

- 7 days
- 15 days
- 30 days
- 60 days
- 90 days
- 180 days
- 1 year

The user can also select which file categories are included in cleanup.

Objectives

- Monitor mobile storage automatically.
- Detect when storage reaches 90%.
- Notify the user about high storage usage.
- Analyze storage after user confirmation.
- Detect duplicate files.
- Detect large files.
- Detect old files.
- Provide user-defined file cleanup.
- Allow users to select the file period .
- Allow users to select file categories for cleanup.
- Identify potentially unused applications.
- Compress supported large files.
- Create ZIP archives.
- Calculate storage savings.
- Display storage recommendations.
- Provide storage reports.
- Store analysis information in a local database.

Scope

The application can be used by:

- Android smartphone users
- Students
- Office users
- Users with limited storage
- Users who frequently download files
- Users who store large media files
- Users who regularly need storage cleanup

The system can later be extended with:

- Scheduled cleanup
- Cloud backup
- Storage history
- Advanced file categories
- Password-protected ZIP files
- Multiple compression levels
- Storage analytics
- Scheduled compression
- File recovery/recycle-bin functionality

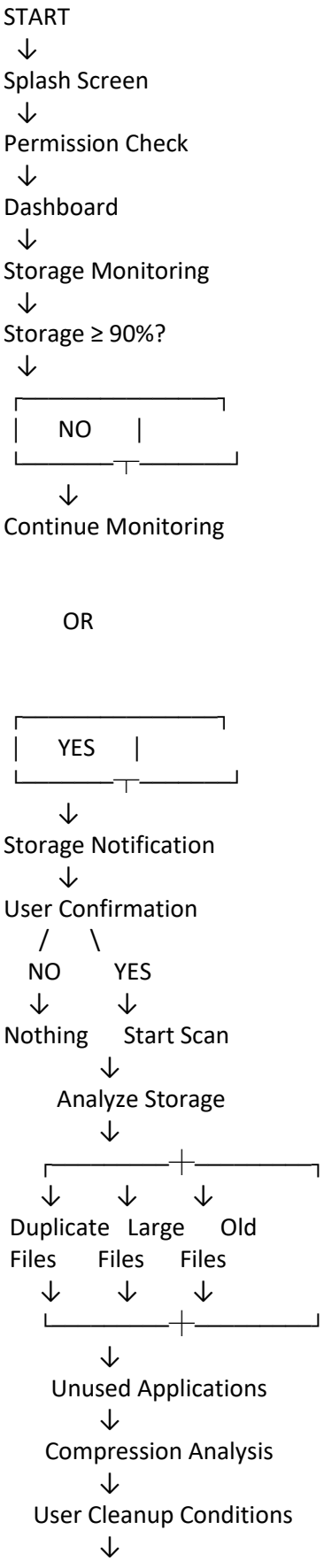
Software Requirements

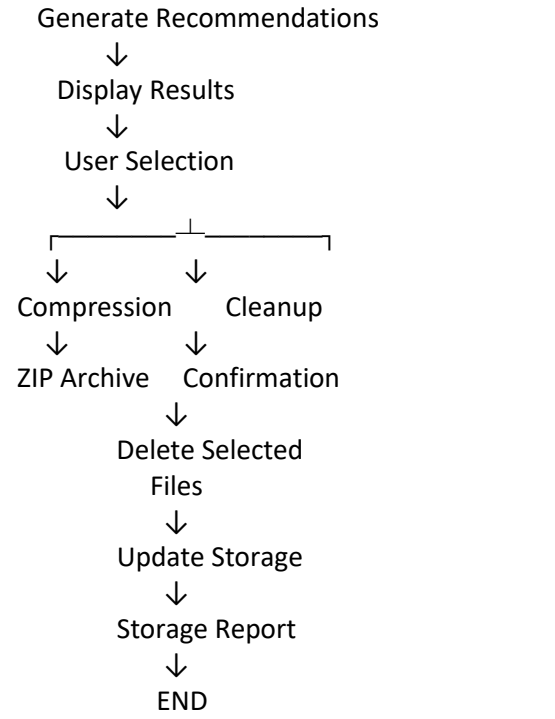
- Android Studio
- Java
- XML
- Android SDK
- Android Storage APIs
- MediaStore API
- UsageStatsManager
- Android Notification APIs
- Java ZIP utilities

Hardware Requirements

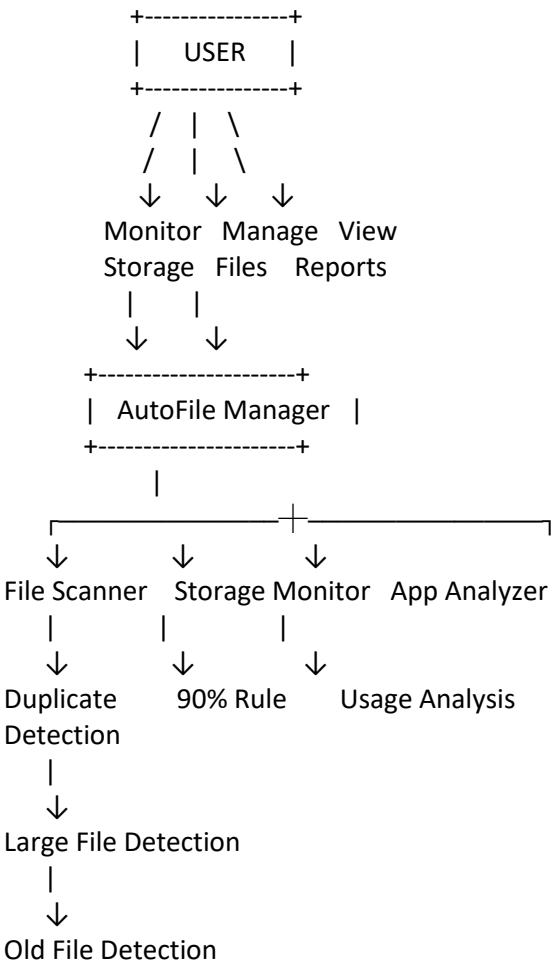
- Windows/Linux/Mac computer
- Minimum 8 GB RAM
- Android smartphone
- USB cable
- Android device for testing
- Internet connection

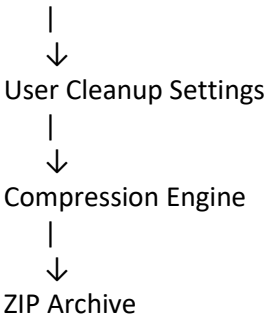
Step 3: Main Project Flow





Step 4: Use Case Diagram

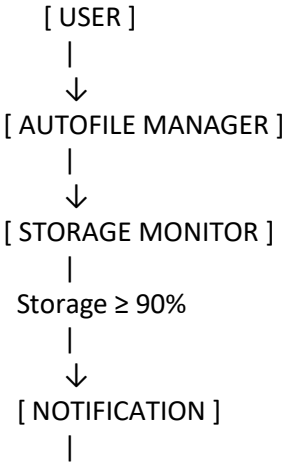


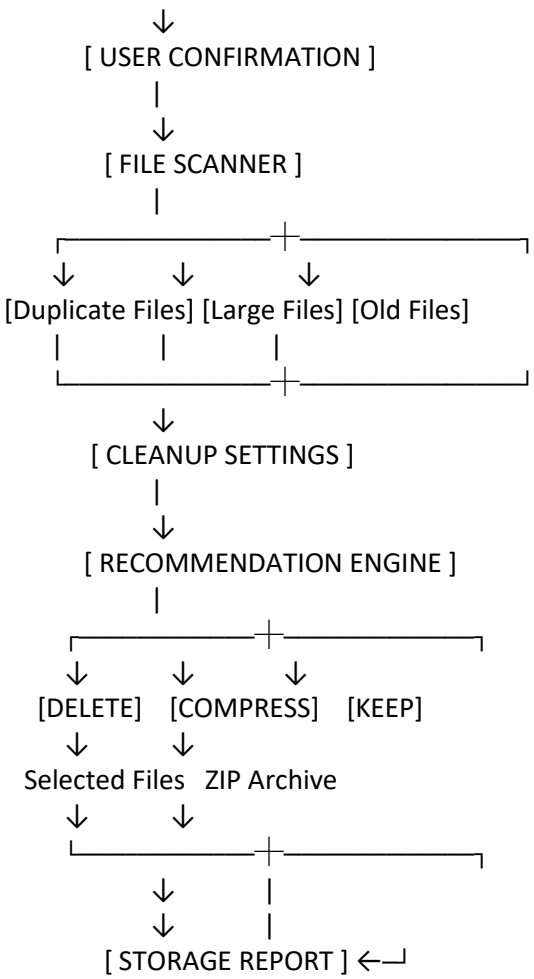


Main Use Cases

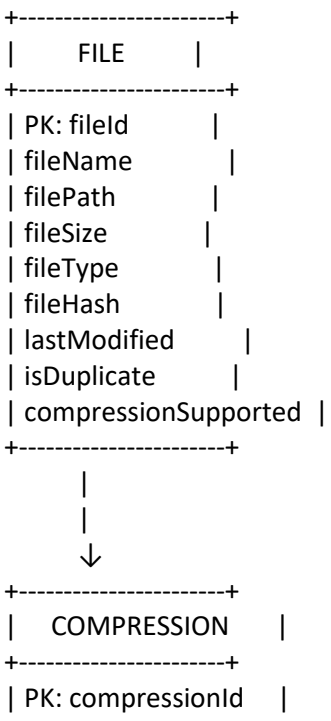
1. Monitor Storage
2. Receive Storage Notification
3. Confirm Storage Scan
4. Scan Files
5. Detect Duplicate Files
6. Detect Large Files
7. Detect Old Files
8. Configure Cleanup Conditions
9. Delete Old Files
10. Analyze Unused Applications
11. Analyze Compression
12. Compress Files
13. Create ZIP Archives
14. Calculate Storage Savings
15. View Recommendations
16. Select Cleanup Files
17. Confirm Cleanup
18. View Storage Report

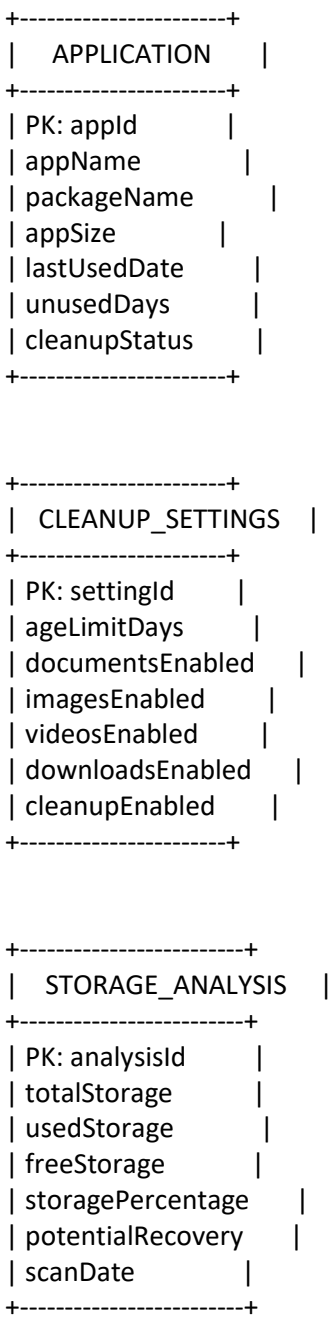
Step 5: Data Flow Diagram (DFD)





Step 6: ER Diagram





Step 7: Database Design

File Table

Columns / Fields	Type	Description
fileId	Integer	Primary Key
fileName	Text	File name
filePath	Text	File location
fileSize	Long	File size
fileType	Text	File type

Columns / Fields	Type	Description
fileHash	Text	File hash
lastModified	Long	Last modification date
isDuplicate	Boolean	Duplicate status
compressionSupported	Boolean	Compression status

Compression Table

Field	Type	Description
compressionId	Integer	Primary Key
fileId	Integer	Foreign Key
originalSize	Long	Original file size
compressedSize	Long	Compressed file size
compressionRatio	Double	Compression ratio
spaceSaved	Long	Space saved
archivePath	Text	ZIP location
compressionDate	Long	Compression date

Application Table

Field	Type	Description
appId	Integer	Primary Key
appName	Text	Application name
packageName	Text	Android package
appSize	Long	Application size
lastUsedDate	Long	Last used date
unusedDays	Integer	Days unused
cleanupStatus	Text	Application status

Cleanup Settings Table

Field	Type	Description
settingId	Integer	Primary Key
ageLimitDays	Integer	User-selected file age
documentsEnabled	Boolean	Document cleanup
imagesEnabled	Boolean	Image cleanup
videosEnabled	Boolean	Video cleanup
downloadsEnabled	Boolean	Download cleanup
cleanupEnabled	Boolean	Cleanup status

Storage Analysis Table

Field	Type	Description
analysisId	Integer	Primary Key
totalStorage	Long	Total storage
usedStorage	Long	Used storage
freeStorage	Long	Free storage
storagePercentage	Double	Storage usage
potentialRecovery	Long	Potential storage recovery
scanDate	Long	Analysis date

Step 8: Android Screens

1. Splash Screen

+--------------------------------+

AUTOFILE MANAGER

[LOGO]

Automated Storage Manager

Loading...

+--------------------------------+

---

2. Permission Screen

+--------------------------------+

PERMISSION REQUIRED

AutoFile Manager needs access
to analyze device storage.

[CONTINUE]

+--------------------------------+

---

3. Dashboard

+--------------------------------+

AUTOFILE MANAGER

Storage: 91% Used



Used: 116 GB
Free: 12 GB

Potential Recovery
7.4 GB

+--------------------------------+

Duplicate Files
1.8 GB

Large Files
2.4 GB

Old Files
900 MB

Unused Apps
2.3 GB

Compressible Files
1.6 GB

[SCAN NOW]

[CLEANUP SETTINGS]

+--------------------------------+

---

4. Storage Notification

When storage reaches 90%:

+--------------------------------+

⚠ STORAGE ALMOST FULL

Your device storage is 91% full.

AutoFile Manager can analyze
your storage.

[SCAN NOW]

[NOT NOW]

+--------------------------------+

---

5. Cleanup Settings Screen

The user selects the period of files for cleanup.

+--------------------------------+

CLEANUP SETTINGS

Delete files older than:

○ 7 Days

○ 15 Days

● 30 Days

○ 60 Days

○ 90 Days

○ 180 Days

○ 1 Year

+--------------------------------+

File Categories

☑ Documents
☑ Images
☑ Videos
☑ Downloads

+--------------------------------+

Automatic Cleanup

[ON / OFF]

[SAVE SETTINGS]

+--------------------------------+

---

6. Custom Cleanup Condition

The user can select a predefined period.

Example:

Selected Condition:

Files older than 60 days

The system calculates:

Current Date
↓
Current Date - 60 Days
↓
File Last Modified Date
↓
Older Than Condition?
↓
Add To Cleanup List

---

7. Old Files Screen

+--------------------------------+

OLD FILES

Condition:
Files older than 60 days

☑ OldVideo.mp4 850 MB
☑ Backup.zip 450 MB
☐ Document.pdf 25 MB
☑ OldPhotos 300 MB

Selected:
1.6 GB

Potential Storage Saving:
1.6 GB

[REVIEW]

[DELETE SELECTED]

+--------------------------------+

---

8. Cleanup Confirmation Screen

+--------------------------------+

CONFIRM CLEANUP

Condition:
Files older than 60 days

Selected Files:
3

Total Size:
1.6 GB

Files selected for deletion:

OldVideo.mp4
Backup.zip
OldPhotos

[CANCEL]

[CONFIRM DELETE]

+---------------------------------+

---

9. Cleanup Result Screen

+---------------------------------+

CLEANUP COMPLETE

Files Removed:
3

Storage Recovered:
1.6 GB

Previous Storage:
91%

Current Storage:
90%

[DONE]

+---------------------------------+

---

10. Duplicate Files Screen

+---------------------------------+

DUPLICATE FILES

18 duplicate files found.

☑ IMG_1023.jpg 8 MB
☑ IMG_1023_copy.jpg 8 MB

☑ Video01.mp4 250 MB
☑ Video01(1).mp4 250 MB

Potential Saving:
266 MB

[REVIEW]

[DELETE SELECTED]

+----------+

---

11. Large Files Screen

+----------+

LARGE FILES

Video.mp4 1.8 GB
Movie.mp4 1.2 GB
Backup.zip 850 MB
Project.pdf 620 MB

Total:
4.47 GB

[REVIEW FILES]

+----------+

---

12. Compression Screen

+----------+

COMPRESS FILE

ProjectBackup.dat

Original Size:
1.2 GB

Compression Supported:
YES

Estimated Compressed Size:
650 MB

Potential Saving:
550 MB

[COMPRESS]

[CANCEL]

+---------------------------------+

---

13. Compression Result Screen

+---------------------------------+

COMPRESSION COMPLETE

Original Size:
1.2 GB

Compressed Size:
650 MB

Space Saved:
550 MB

Compression:
45.8%

Archive:
AutoFile Manager/Compressed/
ProjectBackup.zip

[OPEN ZIP]

[DONE]

+---------------------------------+

---

14. Unused Applications Screen

+---------------------------------+

UNUSED APPS

Game XYZ

Last Used:
45 days ago

Size:
2.3 GB

Shopping App

Last Used:
38 days ago

Size:
750 MB

App XYZ

Last Used:
61 days ago

Size:
1.2 GB

[REVIEW]

[UNINSTALL]

+--------------------------------+

---

15. Storage Recommendation Screen

+--------------------------------+

STORAGE RECOMMENDATIONS

Duplicate Files 1.8 GB
Large Files 2.4 GB
Old Files 900 MB
Unused Apps 2.3 GB
Compression 550 MB

Potential Recovery:
7.95 GB

☑ Duplicate Files
☑ Large Files
☐ Old Files

☐ Unused Apps
☐ Compression

[REVIEW]

+--------------------------------+

---

16. Storage Report Screen

+--------------------------------+

STORAGE REPORT

Total Storage:
128 GB

Used Storage:
116 GB

Free Storage:
12 GB

Storage Usage:
91%

Duplicate Files:
1.8 GB

Large Files:
2.4 GB

Old Files:
900 MB

Unused Apps:
2.3 GB

Compression Saving:
550 MB

Potential Recovery:
7.95 GB

[SCAN AGAIN]

+--------------------------------+

Step 9: Project Folder Structure

```
app
|
├── src/main/java/com/autofilemanager
|
|   ├── activities
|   |   SplashActivity.java
|   |   MainActivity.java
|   |   DashboardActivity.java
|   |   DuplicateActivity.java
|   |   LargeFilesActivity.java
|   |   OldFilesActivity.java
|   |   CleanupSettingsActivity.java
|   |   CleanupConfirmationActivity.java
|   |   CleanupResultActivity.java
|   |   UnusedAppsActivity.java
|   |   CompressionActivity.java
|   |   CompressionResultActivity.java
|   |   StorageReportActivity.java
|   |
|   ├── database
|   |   AppDatabase.java
|   |   FileDao.java
|   |   AppDao.java
|   |   StorageDao.java
|   |   CompressionDao.java
|   |   CleanupSettingsDao.java
|   |
|   ├── model
|   |   FileModel.java
|   |   AppModel.java
|   |   StorageModel.java
|   |   CompressionModel.java
|   |   CleanupSettings.java
|   |
|   ├── scanner
|   |   FileScanner.java
|   |   DuplicateScanner.java
|   |   LargeFileScanner.java
|   |   OldFileScanner.java
|   |
|   ├── analyzer
|   |   StorageAnalyzer.java
|   |   AppUsageAnalyzer.java
|   |   CleanupAnalyzer.java
|   |   CompressionAnalyzer.java
|   |
|   ├── compression
|   |   ZipCompressor.java
|   |   CompressionManager.java
```

```
│   │
│   ├── services
│   │   StorageMonitorService.java
│   │   CleanupService.java
│   │
│   ├── notification
│   │   StorageNotification.java
│   │
│   ├── utils
│   │   HashUtils.java
│   │   FileUtils.java
│   │   StorageUtils.java
│   │   CompressionUtils.java
│   │   DateUtils.java
│   │
│   └── adapter
│       FileAdapter.java
│       AppAdapter.java
│       CompressionAdapter.java
│
└── src/main/res
    │
    ├── layout
    │   activity_splash.xml
    │   activity_main.xml
    │   activity_dashboard.xml
    │   activity_cleanup_settings.xml
    │   activity_old_files.xml
    │   activity_cleanup_confirmation.xml
    │   activity_cleanup_result.xml
    │   activity_duplicate.xml
    │   activity_large_files.xml
    │   activity_unused_apps.xml
    │   activity_compression.xml
    │   activity_compression_result.xml
    │   activity_storage_report.xml
    │
    ├── drawable
    │
    ├── mipmap
    │
    └── values
        colors.xml
        strings.xml
        themes.xml
```

Step 10: Development Process

Phase 1 – Project Setup

1. Install Android Studio.
2. Create a new Android project.
3. Select Java.
4. Configure Android SDK.
5. Create project package structure.

Phase 2 – UI Development

6. Design Splash Screen.
7. Design Permission Screen.
8. Design Dashboard.
9. Design Cleanup Settings.
10. Design Old Files screen.
11. Design Cleanup Confirmation screen.
12. Design Cleanup Result screen.
13. Design Duplicate Files screen.
14. Design Large Files screen.
15. Design Unused Apps screen.
16. Design Compression screen.
17. Design Compression Result screen.
18. Design Storage Report.

Phase 3 – Storage Monitoring

19. Calculate total storage.
20. Calculate used storage.
21. Calculate free storage.
22. Calculate storage percentage.
23. Implement 90% threshold.
24. Implement storage notification.
25. Implement scan confirmation.

Phase 4 – File Analysis

26. Implement file scanning.
27. Implement duplicate detection.
28. Implement file hashing.
29. Implement large file detection.
30. Implement old file detection.
31. Implement file age calculation.

Phase 5 – User-Defined Cleanup

32. Create Cleanup Settings screen.
33. Add predefined period conditions.
34. Add 7-day option.
35. Add 15-day option.
36. Add 30-day option.
37. Add 60-day option.
38. Add 90-day option.
39. Add 180-day option.

40. Add 1-year option.
41. Add file category selection.
42. Store selected cleanup conditions.
43. Scan files according to selected condition.
44. Display matching files.
45. Calculate total selected size.
46. Display cleanup confirmation.
47. Delete selected files.
48. Update storage information.

Phase 6 – Application Analysis

49. Implement application usage analysis.
50. Calculate unused days.
51. Identify applications unused for the selected period.
52. Display unused applications.
53. Start Android uninstall flow.

Phase 7 – Compression

54. Implement compression analysis.
55. Detect supported file types.
56. Create ZIP archives.
57. Calculate original size.
58. Calculate compressed size.
59. Calculate compression ratio.
60. Calculate storage saved.
61. Store ZIP location.
62. Display compression result.

Phase 8 – Database

63. Implement Room Database.
64. Create File Entity.
65. Create Application Entity.
66. Create Compression Entity.
67. Create Cleanup Settings Entity.
68. Create Storage Analysis Entity.
69. Implement DAO classes.
70. Store analysis results.

Phase 9 – Recommendations

71. Calculate potential storage recovery.
72. Display duplicate recommendations.
73. Display large file recommendations.
74. Display old file recommendations.
75. Display unused application recommendations.
76. Display compression recommendations.

Phase 10 – Reports

77. Create storage report.
78. Display storage usage.
79. Display potential recovery.
80. Display cleanup history.
81. Display compression savings.

Phase 11 – Testing

82. Test permissions.
83. Test storage monitoring.
84. Test notification.
85. Test scan confirmation.
86. Test duplicate detection.
87. Test large file detection.
88. Test old file detection.
89. Test user-defined cleanup.
90. Test file category filtering.
91. Test file deletion.
92. Test unused application detection.
93. Test compression.
94. Test ZIP creation.
95. Test database.
96. Test storage report.
97. Test application on Android device.
98. Build final APK.

Step 11: Testing

Test Case	Expected Result
Launch application	Splash screen appears
Open Dashboard	Storage information appears
Permission granted	Storage features become available
Permission denied	Application handles permission state
Storage below 90%	Monitoring continues
Storage reaches 90%	Notification appears
User selects Not Now	Scan does not start
User selects Scan Now	Storage scan starts
Duplicate files exist	Duplicate files are detected
Large files exist	Large files are displayed
Old files exist	Old files are displayed
User selects 30 days	Files older than 30 days are identified
User selects 60 days	Files older than 60 days are identified
User selects 90 days	Files older than 90 days are identified
User selects 1 year	Files older than 1 year are identified
User selects Images	Image files are included

Test Case	Expected Result
User selects Videos	Video files are included
User selects Documents	Document files are included
User selects Downloads	Download files are included
User disables category	Category is excluded
Cleanup settings saved	Settings are stored
Cleanup list generated	Matching files are displayed
User cancels cleanup	Files remain unchanged
User confirms cleanup	Selected files are removed
Storage updated	New storage value is displayed
Compression selected	Compression starts
ZIP created	ZIP archive is created
Compression completed	Storage savings are displayed
Unused application detected	Application appears in list
Uninstall selected	Android uninstall flow starts
Database updated	Latest information is stored
Scan repeated	New analysis is generated
Application restarted	Stored settings remain available

Step 12: Main Java Logic

1. Storage Monitoring

Get Total Storage
↓
Get Available Storage
↓
Calculate Used Storage
↓
Calculate Storage Percentage
↓
Storage ≥ 90%?
/ \
YES NO
↓ ↓
Notification Continue
↓
User Confirmation
↓
Start Scan

---

2. Duplicate Detection

Scan Files
↓

Get File Size
↓
Group Same-Sized Files
↓
Calculate Hash
↓
Compare Hash
↓
Same Hash?
/ \
YES NO
↓ ↓
Duplicate Unique

---

3. Large File Detection

Scan Files
↓
Get File Size
↓
Compare With Large File Threshold
↓
Large?
/ \
YES NO
↓ ↓
Show Ignore

---

4. User-Defined Old File Detection

User Selects Age
↓
7 / 15 / 30 / 60 / 90 / 180 / 365 Days
↓
Get Current Date
↓
Calculate Cutoff Date
↓
Read File Last Modified Date
↓
File Older Than Cutoff?
/ \
YES NO
↓ ↓
Cleanup List Ignore
↓
Display Files

---

5. Cleanup Category Logic

User Selects Categories
↓
Documents
Images
Videos
Downloads
↓
File Scanner
↓
Check File Category
↓
Category Selected?
/ \
YES NO
↓ ↓
Add File Ignore
↓
Check File Age

---

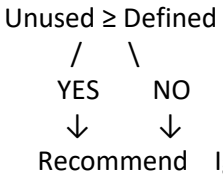
6. Cleanup Logic

Selected Files
↓
Calculate Total Size
↓
Display Cleanup List
↓
User Confirmation
↓
Delete Selected Files
↓
Calculate Recovered Space
↓
Update Database
↓
Update Storage
↓
Display Cleanup Result

---

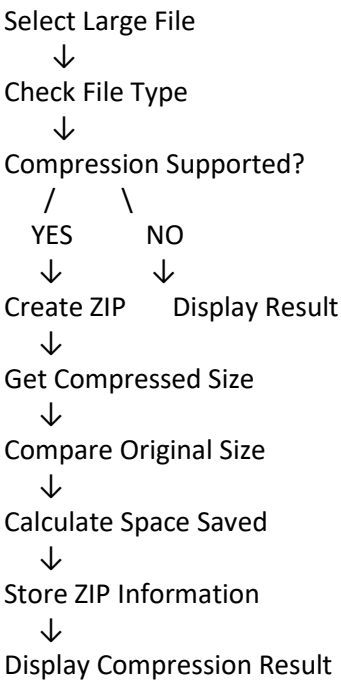
7. Unused Application Detection

Get Application Usage Information
↓
Get Last Used Date
↓
Calculate Unused Days
↓

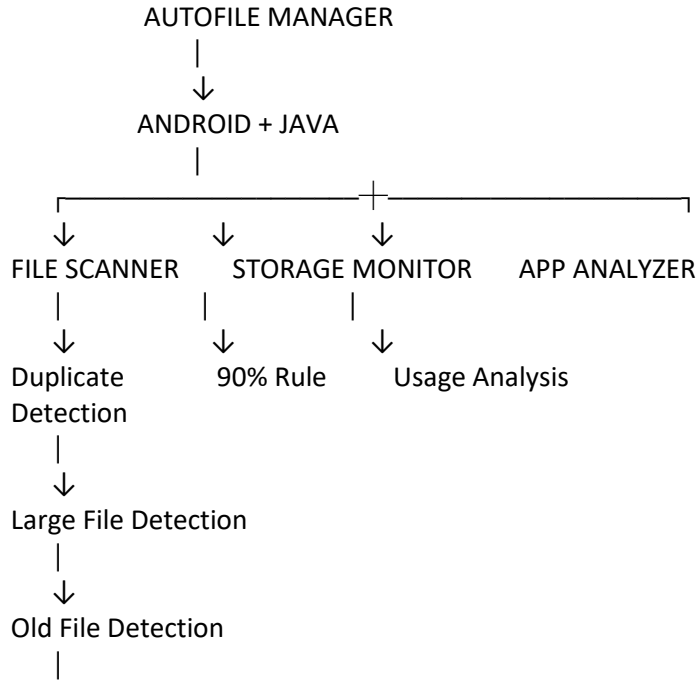


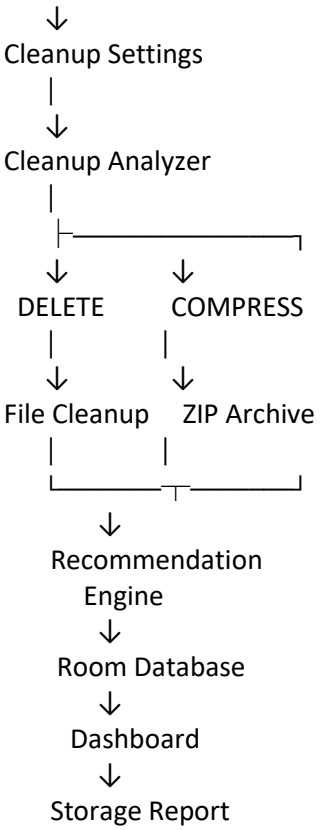
---

8. Compression Logic



Step 13: Project Architecture





Step 14: Project Features

Core Features

1. Automated storage monitoring
2. 90% storage threshold
3. Storage notifications
4. User confirmation before scanning
5. Duplicate file detection
6. Large file detection
7. Old file detection
8. User-defined cleanup age
9. 7-day cleanup condition
10. 15-day cleanup condition
11. 30-day cleanup condition
12. 60-day cleanup condition
13. 90-day cleanup condition
14. 180-day cleanup condition
15. 1-year cleanup condition
16. File category selection
17. Document cleanup
18. Image cleanup
19. Video cleanup
20. Download cleanup
21. Cleanup confirmation
22. Selected file deletion

23. Unused application detection
24. Application usage analysis
25. Application uninstall flow
26. Large file compression
27. ZIP archive creation
28. Compression result calculation
29. Storage saving calculation
30. Cleanup recommendations
31. Storage reports
32. Room Database
33. Background storage monitoring
34. Notification system

Step 15: User-Defined Cleanup Feature

The user can control the cleanup condition.

Example 1

User selects:

Delete files older than 30 days

The application identifies files whose last modification date is older than 30 days.

Example 2

User selects:

Delete files older than 90 days

The application identifies files whose last modification date is older than 90 days.

Example 3

User selects:

Delete files older than 1 year

The application identifies files whose last modification date is older than 365 days.

File Category Selection

Documents
☑

Images
☑

Videos
☐

Downloads
☑

The cleanup scanner processes only the selected categories.

Step 16: File Compression Module

The compression module analyzes large files and creates ZIP archives.

Example:

Original File

ProjectBackup.dat
1.2 GB

↓

Compression

↓

ProjectBackup.zip
650 MB

↓

Space Saved

550 MB

Compression Location

Internal Storage
│
└── AutoFile Manager
    │
    ├── Compressed
    │   ├── ProjectBackup.zip
    │   ├── Documents.zip

```
│    └── Backup.zip
│
└── Reports
    ├── StorageReport.pdf
    └── CleanupReport.pdf
```

Step 17: Storage Calculation

Storage Usage

Used Storage =
Total Storage - Free Storage

Storage Percentage

Storage Percentage =
(Used Storage / Total Storage) × 100

Potential Recovery

Potential Recovery =
Duplicate Files
+
Old Files
+
Selected Large Files
+
Other Selected Files

Compression Saving

Space Saved =
Original Size - Compressed Size

Step 18: Project Innovation

AutoFile Manager combines multiple storage-management functions into one Android application.

The application provides:

Storage Monitoring
↓
Automatic Notification
↓
User Confirmation
↓
Storage Analysis
↓
Duplicate Detection

↓
Large File Detection
↓
User-Defined Old File Detection
↓
Unused Application Detection
↓
Compression Analysis
↓
Storage Recommendations
↓
User Selection
↓
Cleanup / Compression
↓
Storage Report

The user-defined cleanup condition allows the application to adapt to different storage-management preferences.

Step 19: Final Project Description

AutoFile Manager – Automated Android File & Storage Management System is an Android application developed using Java that monitors device storage and provides automated file-management functionality.

The application monitors storage usage and generates a notification when storage reaches 90% or above. After confirmation, the application analyzes device storage and identifies duplicate files, large files, old files and potentially unused applications.

The application provides a user-defined cleanup system in which users can select the age of files to be included in cleanup. Available conditions include 7 days, 15 days, 30 days, 60 days, 90 days, 180 days and 1 year.

Users can also select file categories such as documents, images, videos and downloads.

The application analyzes large files for compression and creates ZIP archives for supported files. Compression results include original size, compressed size and storage saved.

The system displays storage recommendations, allows users to select files for cleanup or compression, and provides storage reports after operations.

Step 20: Technologies Used

Category	Technology
Programming Language	Java

Category	Technology
Platform	Android
IDE	Android Studio
UI	XML
Database	Room / SQLite
Storage	Android Storage APIs
Media Files	MediaStore
App Usage	UsageStatsManager
Compression	Java ZIP
Notifications	Android Notification APIs
Background Processing	Android background mechanisms
Architecture	Activity/Fragment + Database/Repository
Version Control	Git/GitHub

Final Project Title

AutoFile Manager

Automated Android File & Storage Management System

Technology Stack

Java | Android Studio | XML | Android SDK | Jetpack | Room Database | Storage APIs | MediaStore | UsageStatsManager | Java ZIP | Notifications

Core Workflow

Monitor → Notify → Confirm → Analyze → Detect → Configure → Recommend → Clean/Compress → Confirm → Execute → Report

----------xxxxxxxx----------
-=